

Лабораторна робота №5. Локальне розгортання, навантаження і безпека

25 серпня 2026 р.

Мета

Розгорнути локальну мовну модель, виміряти її поведінку під навантаженням і показати експериментально, які захисти від prompt injection працюють, а які лише виглядають працюючими.

Робота спирається на лекції 9, 10, 18. Аудиторний час — 3 години, самостійна робота — 6 годин.

Теоретичні відомості

Генерація складається з двох фаз різної природи. Префіл обробляє весь запит одразу і впирається в обчислення; декод породжує токени по одному і впирається в пропускну здатність пам'яті, бо на кожен токен доводиться перерахувати ваги. Звідси й дві різні метрики: час до першого токена (TTFT) залежить від довжини запиту, а час на наступний токен (TPOT) — від пам'яті й розміру моделі.

- **Наскрізна затримка складається з двох частин:**

$$T = \text{TTFT} + \text{TPOT} \cdot N_{\text{out}}$$

Формула стандартна для інструментів вимірювання; якщо TPOT визначено строго як середній інтервал *між* токенами, точнішим є множник $N_{\text{out}} - 1$, бо перший токен уже врахований у TTFT. На довгих відповідях різниця несуттєва, на коротких — ні, тому в звіті вкажіть, яке означення ви використали.

- **KV-cache росте лінійно з довжиною і батчем:**

$$S_{\text{KV}} = 2 \cdot L \cdot n_{kv} \cdot d_h \cdot b \cdot s \cdot B$$

де L — шарів, n_{kv} — KV-голів, d_h — вимір голови, b — байтів на число, s — довжина, B — батч. Саме залишок пам'яті під кеш, а не заявлена довжина контексту, задає кількість одночасних сеансів.

- **Квантизація торгує якістю за пам'ять.** Для моделі класу 7B чотири біти коштують близько двох відсотків perplexity, три біти — понад десять, і б'ють передусім по багатокрокових обчисленнях. Числа зняті на конкретному наборі й конкретним методом, тому свою втрату якості міряють на своїх задачах, а не переносять із чужої таблиці.
- **Недовірений текст стає інструкцією.** У звичайному застосунку потік керування зафіксований у коді, а недовірений ввід є даними для парсера. В агенті потік керування породжується з тексту, і текст цей приходить із документів, з відповідей інструментів і від інших агентів. Пряма ін'єкція живе в запиті користувача, непряма — у документі, який потрапляє в контекст через пошук; контролює її той, хто написав документ.
- **Системний промпт не є межею безпеки.** Виміряна ефективність промптових захистів падає під адаптивною атакою: атакувальник ходить другим і бачить ваш захист. Тому єдиний надійний важіль архітектурний — обмежити *наслідок*, а не вхід: заборонити запис без підтвердження людини, розділити права інструментів, розірвати смертельну триаду (приватні дані, недовірений текст, канал назовні).
- **Двадцять сценаріїв — це пілот, а не вимірювання.** Одна атака важить п'ять відсоткових пунктів, тому висновок формулюється як напрям, а не як виміряна ефективність захисту. Для кожного наслідку критерій успіху атаки має бути машинно перевірюваним, інакше підрахунок стане предметом суперечки.

Корисні посилання для ознайомлення:

- Kwon W. та ін. [Efficient Memory Management for LLM Serving with PagedAttention](#), SOSP 2023;
- [llama.cpp server](#), [Ollama FAQ](#) та [vLLM engine arguments](#);
- Greshake K. та ін. [Not What You've Signed Up For: Indirect Prompt Injection](#), AISec 2023;
- OWASP GenAI Security Project, [OWASP Top 10 for LLM Applications](#);
- DeBenedetti E. та ін. [Defeating Prompt Injections by Design \(CaMeL\)](#), 2025.

Порядок виконання

Позначка *ауд.* означає, що крок виконується на занятті, *сам.* — самостійно. Довгі прогони винесені на домашній етап.

Профіль обладнання. *CPU-профіль* (гарантований): модель 0,5–1,5В, контекст 2к, конкурентність 1–4. *GPU-профіль*: 7–8В, контекст 8к, конкурентність до 8. Режим *stub* придатний лише для перевірки обгортки сервісу (крок 5); для вимірювань потрібна жива модель.

1. **Розгорнути локальну модель** (40 хв, ауд.): через `Ollama`, `llama.cpp` або `vLLM` — залежно від доступного обладнання — у двох квантизаціях (наприклад, 4- і 8-бітній). Зафіксувати конфігурацію обладнання і обраний профіль.
Перевірка перед вимірюваннями: довжина контексту і кількість паралельних запитів виставлені явно. Значення за замовчуванням тихо зіпсують усі подальші числа.
2. **Порахувати бюджет пам'яті** (40 хв, ауд.): за формулами обчислити обсяг ваг і KV-кешу для двох довжин контексту, потім порівняти з фактичним споживанням.
Перевірка: розбіжність пояснена, а не списана на похибку.
3. **Виміряти затримки** (50 хв, ауд.): TTFT, TROT і пропускну здатність при конкурентності 1, 2, 4 і 8 (для CPU-профілю — 1, 2, 4) на однаковому наборі з 20 запитів. Побудувати таблицю p50 і p95.
Перевірка: перед кожним заміром — розігрів, інакше перший запит зіпсує p50.
4. **Порівняти дві квантизації** (50 хв, сам.): за швидкістю і за якістю на 20 задачах з лабораторної 1 або 2. Сформулювати, чи виправдана втрата якості виграшем у ресурсах.
Перевірка: якість міряється на своїх задачах, а не переноситься з чужої таблиці.
5. **Обгорнути модель сервісом** (40 хв, сам.): таймаут-бюджет, кеш відповідей, резервний варіант на випадок недоступності основної моделі. Показати спрацювання кожного механізму окремо.
6. **Скласти сценарії атак** (50 хв, сам.): 20 сценаріїв prompt injection — 10 прямих у запиті користувача і 10 непрямих, захованих у документах, які потрапляють у контекст через пошук. Сценарії мають націлюватись на щонайменше три різні наслідки: розкриття системної інструкції, виклик забороненого інструмента, витік даних у відповідь. Для кожного наслідку записати **машинно перевірюваний** критерій успіху атаки.
Перевірка: непрямі справді потрапляють у контекст через пошук, а не вставлені в запит руками.
7. **Виміряти базову конфігурацію** (30 хв, ауд.): частка успішних атак до захисту, окремо для прямих і непрямих. Без цього числа наступний крок нічого не доводить.
8. **Впровадити три рівні захисту** (60 хв, сам.): розмежування довіреного і недовіреного тексту в промпті, валідація виходу, архітектурне обмеження (запис лише з підтвердженням людини, окремі права на інструмент). Міряти *кожен окремо* відносно базової конфігурації, а вже

потім усі разом: увімкнені послідовно, вони зміщують ефект захисту з порядком додавання.

9. **Пояснити результат** (40 хв, сам.): які сценарії зактив промпт, які лише архітектура, а які не закрилися взагалі. Для останніх запропонувати залишкові контрзаходи.
10. **Здати evidence package**: конфігурацію обладнання і моделі, таблиці затримок і якості, набір з 20 сценаріїв, таблицю успішності атак до і після кожного рівня захисту, схему меж довіри системи і висновок до 250 слів із переліком відомих обмежень. У висновку окремо зазначити, що вибірка з 20 сценаріїв є піотною.

Крок 9 і є результатом роботи. Список того, що *не* закрилося, цінніший за таблицю з нулями: він показує, що ви розумієте межі власної системи.

Контрольні запитання

1. Чому декод упирається в пропускну здатність пам'яті, а не в обчислення?
2. Як зміниться KV-кеш, якщо подвоїти довжину контексту, і як — якщо подвоїти пакет?
3. Чому саме кеш, а не ваги, зазвичай обмежує кількість одночасних запитів?
4. Що конкретно треба виміряти, щоб стверджувати, що квантизація «майже не погіршила якість»?
5. Чому p95 зростає нелінійно за високого завантаження?
6. Чим семантичний кеш небезпечніший за кеш префікса?
7. У чому різниця між прямим і непрямим prompt injection з погляду того, хто контролює текст?
8. Чому системний промпт не є механізмом безпеки?
9. Які сценарії з вашого набору неможливо закрити нічим, крім архітектури?
10. Що входить до evidence package і що саме кожен його елемент доводить?

Література

1. Lin J. та ін. [AWQ: Activation-aware Weight Quantization](#). MLSys, 2024.
2. DeBenedetti E. та ін. [AgentDojo](#). NeurIPS, 2024.
3. Nasr M. та ін. [The Attacker Moves Second](#). arXiv:2510.09023, 2025.
4. NIST AI 600-1. [AI RMF: Generative AI Profile](#), 2024.
5. Xiao T., Zhu J. [Foundations of Large Language Models](#). arXiv:2501.09223, 2025. Розділ 5.